SE EXPERIMENT 3

Aim: Develop Use Case Diagram and Class Diagram for the Smart Parking Management System (SPMS) project using any open-source tool.

Use Case Diagram

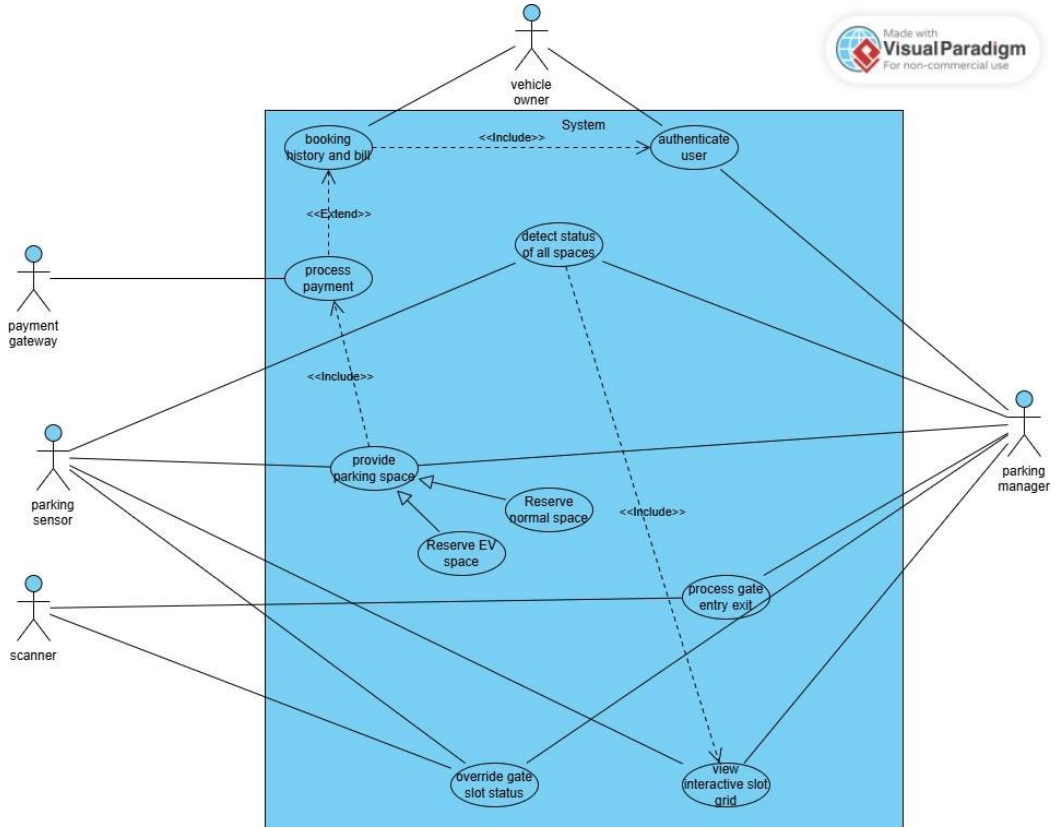
The Use Case Diagram depicts the functional interaction between external actors (human users and hardware/external services) and the system boundary of the Smart Parking Management System.

Actors & Operational Scope:

- Vehicle Owner: Interacts with the system to authenticate credentials, view booking history/bills, and reserve specific parking spaces.
- Parking Manager: Oversees facility operations, views interactive slot grids, detects space statuses, handles entry/exit processing, and manually overrides gate/slot statuses during exceptions.
- Payment Gateway: External service actor responsible for processing payments upon reservation or checkout.
- Parking Sensor: IoT hardware actor supplying status signals to provide real-time parking space data and update space occupancy.
- Scanner: Hardware actor scanning entry/exit passes, connected to gate override and validation flows.

Use Cases & Relationships:

- authenticate user: Core authentication process included (<<Include>>) by booking history and bill.
- booking history and bill: Extended (<<Extend>>) by process payment.
- provide parking space: Generalization parent for Reserve normal space and Reserve EV space. Includes (<<Include>>) process payment.
- detect status of all spaces: Includes (<<Include>>) view (interactive slot) grid for live monitoring.
- process gate entry exit & override gate slot status: Operational management functions connected directly to the Parking Manager, Scanner, and System.



Class Diagram

The Class Diagram models the static structure of the Smart Parking Management System by illustrating system entities, attributes, operational methods, and structural relationships (generalization, association, and dependency).

Class Breakdown & Relationships:

- User (Base Class):
  - *Attributes:* User-id, Name, Email, Password
  - *Method:* Authenticate()
  - *Relationships:* Parent class in a generalization hierarchy with Vehicle Owner and Parking Manager.

- Vehicle Owner (Inherits from User):
  - *Attributes:* Vehicle No.
  - *Methods:* Book_Space(), View History()
  - *Relationships:* Associated with Booking (creates bookings) and Payment.

- Parking Manager (Inherits from User):
  - *Attributes:* Manager-id
  - *Methods:* Detect_Status(), Override_Status()
  - *Relationships:* Associated with Parking Space, Sensor, and Gate for administrative control.

- Parking Space (Base Class):
  - *Attributes:* Space-id, Location, Status
  - *Methods:* Check_Availability(), Update_Status()

- *Relationships:* Parent class generalized into Normal Space and EV Space. Associated with Booking and Sensor.

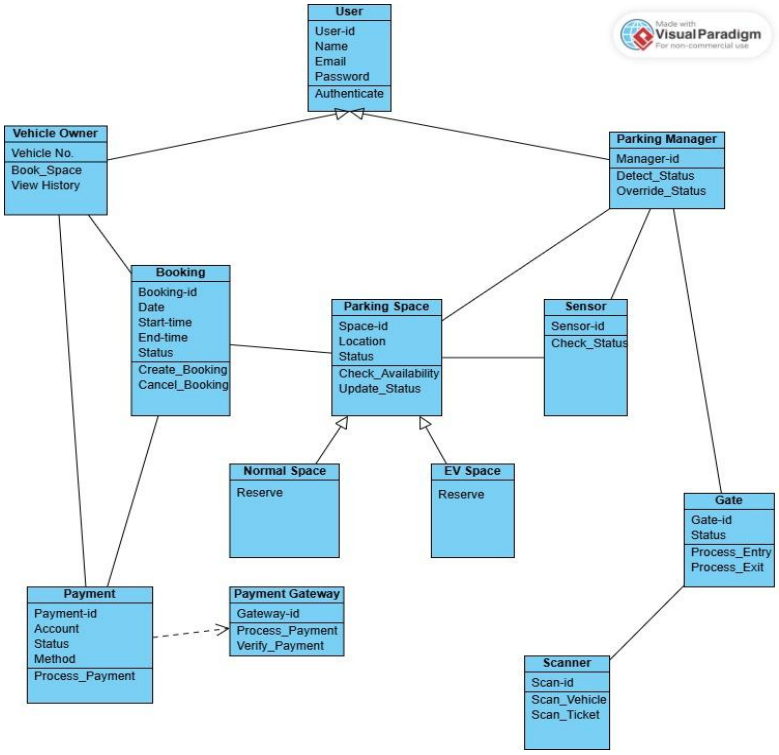
- Normal Space & EV Space (Inherit from Parking Space):
  - *Methods:* Reserve()

- Booking:
  - *Attributes:* Booking-id, Date, Start-time, End-time, Status
  - *Methods:* Create_Booking(), Cancel_Booking()
  - *Relationships:* Links Vehicle Owner with Parking Space and triggers Payment.

- Payment & Payment Gateway:
  - *Payment Attributes:* Payment-id, Account, Status, Method
  - *Payment Method:* Process_Payment()
  - *Payment Gateway Attributes & Methods:* Gateway-id, Process_Payment(), Verify_Payment()
  - *Relationships:* Payment has a dependency relationship (--->) pointing to Payment Gateway.

- Sensor:
  - *Attributes:* Sensor-id
  - *Method:* Check_Status()
  - *Relationships:* Associated with Parking Space and managed by Parking Manager.

- Gate & Scanner:
  - *Gate Attributes & Methods:* Gate-id, Status, Process_Entry(), Process_Exit()
  - *Scanner Attributes & Methods:* Scan-id, Scan_Vehicle(), Scan_Ticket()
  - *Relationships:* Scanner is associated with Gate, which is controlled by Parking Manager.



Result:

The Use Case Diagram and Class Diagram for the Smart Parking Management System (SPMS) were successfully designed and analyzed using UML modeling tools. The diagrams accurately captured system actor interactions, structural class hierarchies, member attributes, methods, and entity relationships required for complete domain modeling.